ICS-344 · INFORMATION SECURITY · TERM 252

DVSA Vulnerability Discovery & Remediation

Damn Vulnerable Serverless Application — full-stack security audit on AWS

12
Vulnerabilities

10
Official Lessons

2
Bonus Findings

TEAM 56 · Mohammed Alroomi · Abdulmalik Al AlShaikh · Naif AlHammad · Faisal Alharbi

AWS · us-east-1 · May 2026

What we audited & how we approached it

THE TARGET

DVSA — Damn Vulnerable Serverless Application

An intentionally vulnerable AWS serverless e-commerce stack used to teach modern cloud security. Built on Lambda, API Gateway, DynamoDB, Cognito and S3.

STACK

- AWS Lambda (Node.js & Python)
- API Gateway, Cognito, DynamoDB
- S3, SES, CloudWatch, IAM
- Region: us-east-1 (N. Virginia)

METHODOLOGY

1. **Recon**
   Map APIs, Lambda functions, IAM roles, S3 buckets
2. **Exploit**
   Reproduce each lesson with working PoC payloads
3. **Evidence**
   Capture CloudWatch logs, requests, screenshots
4. **Fix**
   Patch source code, IAM policies, API Gateway config
5. **Verify**
   Re-run exploits — confirm rejection & no regressions

12 vulnerabilities · severity-ranked

L01 CRITICAL
Event Injection (RCE)

L02 CRITICAL
Broken Authentication (JWT)

L03 HIGH
Sensitive Data Exposure

L04 HIGH
Insecure Cloud Config (S3)

L05 HIGH
Broken Access Control

L06 MEDIUM
Denial of Service

L07 CRITICAL
Over-Privileged Functions

L08 HIGH
Race Condition (Logic)

L09 CRITICAL
Vulnerable Dependencies

L10 MEDIUM
Unhandled Exceptions

B01 HIGH
Second-Order SQL Injection

B02 HIGH
Billing Data in CloudWatch

4 Critical · 6 High · 2 Medium

Unauthenticated RCE via node-serialize

CRITICAL *DVSA-ORDER-MANAGER calls serialize.unserialize() on raw event body before any auth.*

VULNERABILITY

- node-serialize@0.0.4 rehydrates functions via eval()
- Strings prefixed _$$ND_FUNC$$_ execute on deserialize
- Called on event.body and event.headers pre-auth

EXPLOITATION

- POST JSON with action: "_$$ND_FUNC$$_function(){...}()"
- Payload writes /tmp/pwned.txt and reads it back
- Extended payload dumps process.env → STS creds
- Stolen creds query other users' DynamoDB orders

IMPACT

- Arbitrary code execution inside the Lambda
- STS credential theft → account-wide pivot
- No DVSA account required — only the API URL

FIX

- Remove node-serialize entirely
- Replace with safe JSON.parse(event.body)
- Allowlist-validate every incoming field
- Re-test: payload returns invalid input · no eval

JWT Forgery — Full Account Takeover

CRITICAL *Backend trusted JWT payload claims without verifying the cryptographic signature.*

VULNERABILITY

- Lambda decoded JWT but never verified signature
- username and sub were trusted as identity
- Cognito JWKS endpoint was never consulted

EXPLOITATION

- Capture victim's UUID from any valid JWT
- Take attacker's token, swap username/sub for victim's
- Reuse original signature unchanged
- Orders API returns victim's private order list

IMPACT

- Any authenticated user impersonates anyone
- Read & manipulate every other user's orders
- Complete identity-layer collapse

FIX

- Fetch Cognito JWKS, verify with jose.JWS.createVerify
- Validate iss matches Cognito pool, exp not passed
- Cache JWKS for 6 hours to avoid latency hit
- Re-test: forged token → invalid token

Admin Receipt Dump Open to Anyone

HIGH

DVSA-ADMIN-GET-RECEIPT served all users' receipts to any authenticated caller.

VULNERABILITY

- lambda_handler had zero authorization check
- Read year/month/day fields and went straight to S3
- Returned a presigned URL packaging all receipts

EXPLOITATION

- aws lambda invoke with non-admin User B credentials
- Payload: {year:2026, month:04, day:24}
- Receive HTTP 200 + presigned S3 URL
- ZIP reveals every customer's receipt for that date

IMPACT

- PII exposure: names, items, totals, order IDs
- Cross-tenant data leak from a single API call
- Security-by-obscurity proven to fail

FIX

- Read userId at very first line of handler
- Lookup DynamoDB → enforce isAdmin == True
- Return 403 before any S3 operation
- Re-test: non-admin gets 403 · admin still works

Writable S3 Bucket Triggers Lambda

HIGH

Receipts bucket policy let any authed user PUT objects that triggered downstream Lambda.

VULNERABILITY

- S3 bucket allowed PUT from any authenticated session
- ObjectCreated event invoked DVSA-SEND-RECEIPT-EMAIL
- Object key flowed unchecked into OS-level operations

EXPLOITATION

- aws s3 cp ~/empty s3://dvsa-receipts-bucket/.../trigger.raw
- Bucket accepts upload from non-privileged user
- S3 event fires Lambda with attacker-controlled key
- Key parsed as a legitimate receipt path

IMPACT

- Path to command injection in receipt processor
- Indirect Lambda invocation via event-driven chain
- Backend trusts untrusted filenames

FIX

- Bucket policy: Deny PutObject unless principal is receipt role
- Remove any cross-account / open write access
- Validate object names & types in the consumer
- Re-test: cp from user → AccessDenied

Mark Any Order Paid — Without Paying

HIGH

DVSA-ADMIN-UPDATE-ORDERS callable by regular users — financial fraud as a feature.

VULNERABILITY

- Admin route exposed to any authenticated caller
- No role/isAdmin check at function entry
- Order status transitions weren't validated

EXPLOITATION

- Login as regular user, capture JWT from DevTools
- Send admin update with orderStatus = 200 (paid)
- Order flips to paid · no billing record exists
- Goods shipped without payment

IMPACT

- Direct financial loss to the platform
- Bypasses entire billing & payment workflow
- Top-ranked OWASP risk category in practice

FIX (Hardening)

- Verify isAdmin in DynamoDB before any write
- Server-side state-machine for order transitions
- Only billing path may set status = paid
- Re-test: non-admin → 403 · admin still works

Concurrency Ceiling as an Attack Surface

MEDIUM *Billing Lambda's reserved concurrency of 10 saturated by one user.*

VULNERABILITY

- Reserved concurrency = 10 · no rate limiting
- API Gateway has no per-user throttle
- No queue absorbs burst traffic

EXPLOITATION

- Python threading: 20 parallel POSTs to billing
- All 10 slots saturated → TooManyRequestsException
- Concurrent legitimate users hit 500 errors
- Checkout effectively offline for everyone

IMPACT

- Total denial of revenue path
- One user blocks all other users
- Reputation & SLA damage

FIX

- API Gateway usage plan · per-user rate + burst
- WAF rate-based rule on burst patterns
- SQS queue in front of billing Lambda
- Re-test: 21st request → 429 · others succeed

IAM Wildcards = Account-Wide Blast Radius

CRITICAL *DVSA-SEND-RECEIPT-EMAIL role had * on S3, DynamoDB, and AmazonSESFullAccess.*

VULNERABILITY

- Resource: arn:aws:s3:::* and table/*
- AmazonSESFullAccess attached on top
- Lambda role IS the security perimeter

EXPLOITATION

- Print process.env from inside the Lambda
- Extract STS Access Key, Secret, Session Token
- Export creds locally · run aws dynamodb scan
- Full DVSA-ORDERS-DB exfiltrated

IMPACT

- Complete customer-order data breach
- Lateral movement across the entire AWS account
- Wildcards turn one bug into an account compromise

FIX

- Drop wildcard policies & SES full access
- Resource-scope: GetObject on receipts bucket only
- GetItem on DVSA-ORDERS-DB only · ses:SendEmail only
- Verify in IAM Policy Simulator → Denied

Race Condition — Pay for 1, Receive 5

HIGH

Billing read order state · no atomic lock · update mutated cart mid-flight.

VULNERABILITY

- No conditional write or status lock during billing
- TOCTOU between read price and write status
- Serverless concurrency makes this trivial

EXPLOITATION

- Create order with 1 item · capture orderId
- Fire billing & update {items:{1:5}} simultaneously
- Billing charges for 1 · update sets qty to 5
- DynamoDB: 5 items, 1-item payment

IMPACT

- Direct financial loss · per-order fraud
- Inventory discrepancy & accounting drift
- Hard to detect without transactional logs

FIX

- DynamoDB ConditionExpression: orderStatus = 100
- Atomic flip to 'billing-in-progress' (150)
- Use DynamoDB transactions for multi-step writes
- Re-test: ConditionalCheckFailedException · race blocked

node-serialize 0.0.4 — CVE-2017-5941

CRITICAL *Same root cause as L01 — viewed through the supply-chain lens.*

VULNERABILITY

- Critical CVE public since 2017
- No patched version exists upstream
- Package shipped to production unaudited

EXPLOITATION

- npm audit flags critical vulnerability
- _$$ND_FUNC$$_ payload triggers eval()
- Same RCE path as Lesson 01

IMPACT

- Single dependency turns API into RCE endpoint
- Trusted code is third-party code
- Attack surface expands with every install

FIX

- Remove node-serialize from package.json
- Replace all .unserialize() with JSON.parse()
- Add npm audit to CI/CD as a blocking gate
- Re-verify: npm audit → 0 critical

Stack Traces Leak the Backend Map

MEDIUM *Missing fields → Python KeyErrors propagate through API Gateway to the client.*

VULNERABILITY

- Handlers assumed required fields exist
- No try/except around handler logic
- Tracebacks bubble straight to the response

EXPLOITATION

- POST {action:'orders'} → KeyError: orderId leaked
- POST {action:'billing'} → reveals items field path
- Each error names files, line numbers, internals

IMPACT

- Free reconnaissance for attackers
- Implementation details exposed publicly
- Reduced reliability for legitimate users

FIX

- Validate required inputs at handler entry
- Wrap all logic in try/except
- Return generic 4xx/5xx · log details to CloudWatch only
- Re-test: malformed → 400 with safe message

Stored Today, Executes at Billing

HIGH

Payload planted in DynamoDB · executed later by SQLite-backed cart-total Lambda.

VULNERABILITY

- Order item ID stored unchecked in DynamoDB
- DVSA-GET-CART-TOTAL concatenates it into SQL
- Trust boundary forgotten between two stages

EXPLOITATION

- Create order: items = {'1 UNION SELECT ... FROM sqlite_master':1}
- Add shipping → trigger billing
- CloudWatch shows table metadata leaked
- Schema reconnaissance achieved

IMPACT

- Schema disclosure · data exfiltration risk
- Denial-of-wallet via expensive crafted queries
- Hard to detect — no injection at the entry point

FIX

- Allowlist item IDs as integers at order creation
- Parameterize the SQLite query
- Treat all DynamoDB-read values as untrusted
- Re-test: payload rejected with 400

BONUS 02 · SENSITIVE DATA IN LOGS

Card Numbers in CloudWatch

HIGH

DVSA-ORDER-BILLING printed the full event — including ccn, exp, cvv — to stdout.

VULNERABILITY

- print(json.dumps(event)) called on every request
- Lambda stdout auto-forwarded to CloudWatch Logs
- No redaction · persistent storage of PII + PCI

EXPLOITATION

- Submit any normal checkout in the DVSA UI
- Open /aws/lambda/DVSA-ORDER-BILLING log stream
- Plaintext ccn, exp, cvv visible in event payload
- Any IAM principal with logs:GetLogEvents reads it

IMPACT

- PCI-DSS scope violation · regulatory exposure
- Historical logs hold every prior transaction
- Log access blast radius now includes payment data

FIX

- redact_sensitive_fields() helper before logging
- Recursive redact: ccn, cvv, exp, token, password
- Tighten CloudWatch Logs IAM read access
- Re-test: logs show [REDACTED] for all sensitive keys

CLOSING

Key Lessons Learned

01

Defense-in-depth, not single checkpoints

Every Lambda must enforce its own auth, validation, and limits — never trust the layer before it.

02

Least privilege is resource-scoped

table/* is not least privilege. Wildcards on IAM turn any RCE into an account-wide breach.

03

Treat stored data as untrusted

Second-order injections and TOCTOU bugs hide between stages. Validate on read, not just on write.

04

Dependencies are a security surface

npm audit is non-negotiable. A package without a safe version must be removed, not pinned.

05

Logs are part of the trust boundary

Never log raw events. Redact PII/PCI by default. Tighten log read IAM.

06

Verify after every fix

Re-run the original exploit. A patch isn't real until the same payload is rejected.

Thank You

Team 56 — ICS-344 Information Security · KFUPM · Term 252

Mohammed Alroomi · Abdulmalik Al AlShaikh · Naif AlHammad · Faisal Alharbi

AWS · us-east-1 · May 2026